

SEMINAR PROPOSAL TUGAS AKHIR

Implementasi dan Evaluasi Arsitektur Event-Driven Microservices untuk Menjaga Konsistensi Transaksi pada Apotek Multicabang

Raihan Rizki Alfareza

NIM 23051204067 | S1 Teknik Informatika, Fakultas Teknik

Pembimbing: I Made Suartana, S.Kom., M.Kom.



Universitas Negeri Surabaya | 2026

Pencatatan terpisah: rekonsiliasi lambat dan selisih stok berulang

±450

transaksi penjualan
per hari, 3 cabang

1-2 hari

rekonsiliasi
di cabang

4-5 hari

rekonsiliasi
di gudang

Selalu

selisih stok fisik
tiap rekonsiliasi bulanan

Kondisi eksisting

- Kasir mandiri di tiap cabang
- Excel VBA di gudang distribusi
- Tanpa sinkronisasi real-time



Risiko bila sinkronisasi tidak baik

Dua transaksi kasir pada produk yang sama berisiko saling menimpa stok akhir (lost update).

Risiko integrasi; bukan kejadian pada sistem eksisting.

Pertanyaan penelitian: ketika catatan antarunit dihubungkan, apakah transaksi tetap konsisten?

Sumber: wawancara dan observasi dengan pemilik Apotek Bisma, 5 Juli 2026

Lima masalah; perebutan stok diutamakan sebagai skenario uji

1

Rekonsiliasi memakan 1-5 hari kerja

2

Stok sistem tidak cocok dengan stok fisik

3

Integrasi tanpa sinkronisasi: risiko lost update dan oversell

4

Belum ada dasbor monitoring terpadu

5

Konsekuensi tiap arsitektur belum terbukti

Ilustrasi perebutan stok (F5)

Stok awal = 1 unit

Kasir A

Baca stok = 1

Jual 1 unit

Simpan stok = 0

Kasir B

Baca stok = 1

Jual 1 unit

Simpan stok = 0

Oversell: terjual 2 dari stok 1

Lost update: dua pembaruan saling menimpa

Studi terdahulu menjadi landasan; pengujian langsungnya masih terbuka

Arsitektur dan migrasi

Blinowski et al. (2022)
Söylemez et al. (2022)
Faustino et al. (2024)

Monolith unggul pada beban tertentu; konsistensi data dan operasi menjadi tantangan microservices; migrasi berbiaya

Dasar: monolith sebagai acuan

Event dan konsistensi

Rochman & Suartana (2026)
Bação & Guerreiro (2026)

EDA memangkas waktu proses, tetapi penguncian data memicu kemacetan stok

Dasar: Optimistic Concurrency Control

Uji keandalan

Al-Said Ahmad & Andras (2022)
Hui et al. (2025)

Gangguan sengaja efektif menguji ketahanan; hasil butuh aturan acuan yang pasti

Dasar: fault injection dan test oracle

Metodologi ukur

Rodrigues et al. (2025)

Perbandingan butuh beban dan perangkat yang setara

Dasar: lingkungan uji setara

RUANG PENELITIAN

Menguji dampak pemisahan layanan dan proteksi konsistensi data terhadap konsistensi transaksi pada apotek multicabang.

Empat rumusan masalah dijawab oleh empat tujuan yang sejajar

RM1

Bagaimana merancang dan mengimplementasikan event-driven microservices pada apotek multicabang dengan sumber kebenaran data per layanan?

Empat layanan, otoritas data

RM2

Bagaimana kondisi konsistensi data pada masing-masing arsitektur ketika terjadi konkurensi transaksi?

Invariant bisnis, konkurensi (F5)

RM3

Bagaimana kondisi keandalan transaksi pada masing-masing arsitektur ketika terjadi gangguan atau kegagalan proses transaksi?

Fault injection dan pemulihan (F1-F6)

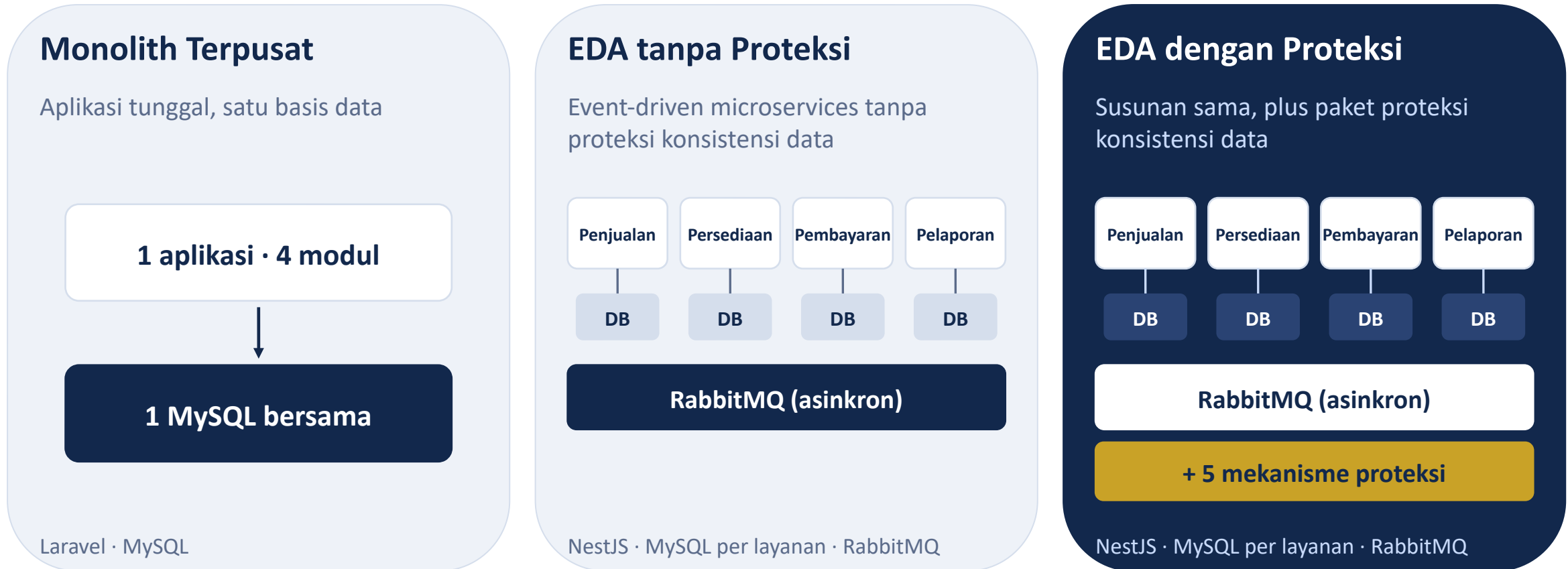
RM4

Bagaimana perbandingan konsistensi data dan keandalan transaksi pada ketiga arsitektur dengan beban kerja dan kondisi pengujian yang setara?

3 kondisi, 3.330 run

Tujuan 1-4 sejajar dengan RM 1-4. Performa dan kompleksitas operasional menjadi dimensi pendukung.

Tiga kondisi arsitektur dibandingkan dengan fungsi bisnis yang sama



EDA vs EDA: hanya paket proteksi yang berbeda. Monolith vs EDA: framework, jumlah basis data, dan komunikasi ikut berbeda, jadi bukan efek satu variabel.

Konsistensi dinilai dari kondisi akhir data, bukan dari selisih sementara

Konsistensi data

kondisi akhir memenuhi aturan bisnis stok dan saldo

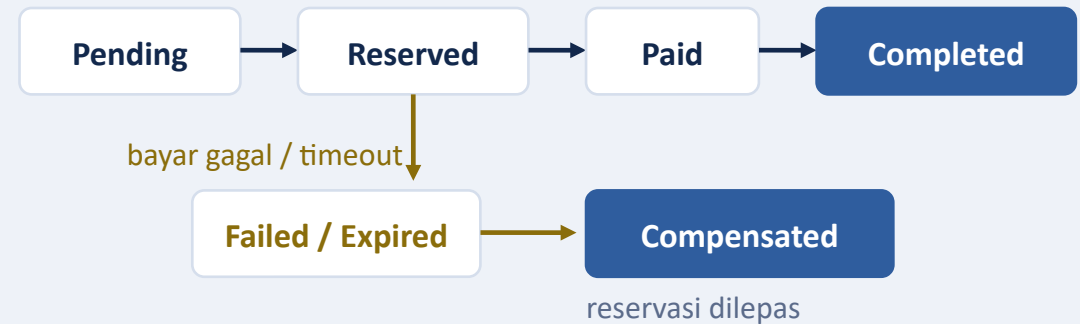
Keandalan transaksi

transaksi selesai sesuai aturan atau pulih ke kondisi yang benar

Empat invariant bisnis

- 1 Available = On-Hand - Reserved
- 2 Available tidak boleh di bawah 0
- 3 Saldo akhir = awal + masuk - terjual
- 4 Status final tidak kembali aktif

Invariant 4: siklus status transaksi



Terlambat = diterima sistem (received_at) lebih dari 30 dtk sejak reservasi. Transaksi final tidak boleh aktif lagi, kapan pun diproses.

Berbeda dari properti Consistency pada ACID, yang hanya berlaku dalam satu transaksi basis data.

Satu data, satu pemilik: empat domain dengan otoritas data yang jelas



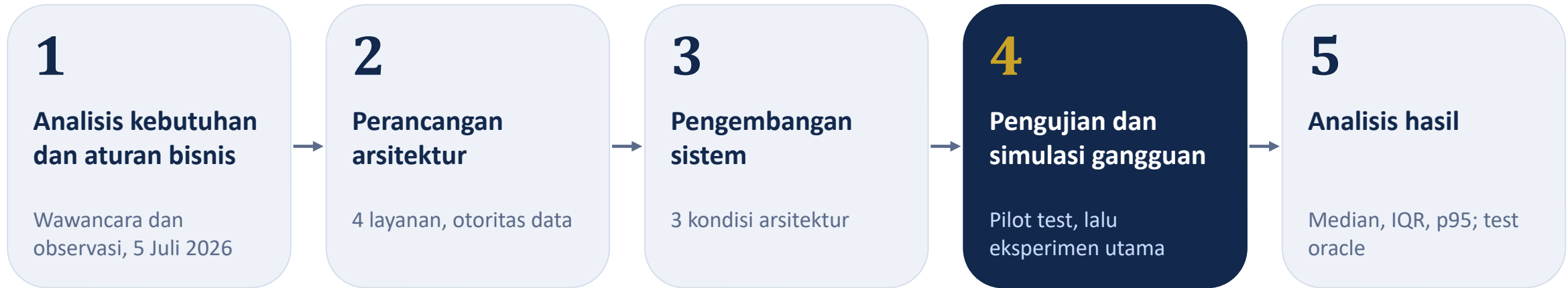
Aturan: data hanya diubah oleh layanan pemiliknya; layanan lain tidak mengubah basis datanya langsung. Berlaku pada ketiga kondisi; pada monolith berupa empat modul.

Lima mekanisme proteksi, masing-masing menargetkan satu risiko

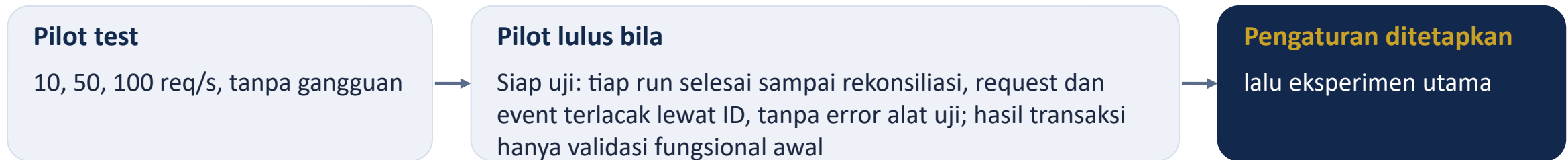


Tanpa Proteksi = tanpa kelima mekanisme ini. Konfigurasi dasar (publisher confirm, antrean durable, ACK manual, 2 consumer) sama di kedua EDA.

Alur penelitian lima tahap; pilot test mendahului eksperimen utama



TAHAP 4: PILOT, LALU EKSPERIMEN UTAMA



Lingkungan: Laravel, NestJS, MySQL InnoDB, RabbitMQ, Docker Compose. Satu host, total CPU dan RAM setara, data awal sintetis dan direset tiap run.

Kriteria integritas bersifat non-kompensatori; performa tidak menutupinya

oversell

= 0

unit terjual melebihi stok

lost update

= 0

pembaruan sah tertimpa

duplicate effect

= 0

satu event, efek bisnis berulang

permanent mismatch

= 0

data lintas layanan tak sesuai setelah pemulihan

untraceable event

= 0

event tak dapat dilacak pada log setelah pemulihan

terminal-state coverage

= 100%

transaksi mencapai status akhir sah

compensation success

= 100%

transaksi yang perlu kompensasi berhasil dipulihkan

read-model lag

≤ 60 dtk

jeda data hingga tampil di Pelaporan

DIMENSI PENDUKUNG, DILAPORKAN TERPISAH

Performa: latency, throughput, technical error rate, CPU, RAM

Kompleksitas operasional: komponen aktif, langkah dan waktu pemulihan manual, intervensi manual

Enam kode gangguan; F1-F4a hanya berlaku pada kedua EDA

EMPAT SKENARIO

Penjualan Bersamaan

Mutasi Stok

Pembayaran Digital

QRIS Statis

Kode	Gangguan	Skenario	Berlaku	Bukti utama
F1	Producer dihentikan setelah commit, sebelum publish	Mutasi Stok	2 EDA	untraceable event, recovery time
F2	Event terkirim ganda (event_id sama)	Mutasi Stok, Pembayaran Digital	2 EDA	duplicate effect
F3	Consumer berhenti sebelum ACK	Mutasi Stok	2 EDA	permanent mismatch, duplicate effect, recovery time
F4a	Pengiriman event ditunda 10 detik	Mutasi Stok	2 EDA	read-model lag, recovery time, permanent mismatch
F4b	Konfirmasi bayar melewati 30 detik	Pembayaran Digital, QRIS Statis	3 kondisi	terminal-state coverage, permanent mismatch
F5	20 pembaruan stok serentak	Penjualan Bersamaan, Mutasi Stok	3 kondisi	oversell, lost update, saldo akhir
F6	Bayar gagal setelah stok direservasi	Pembayaran Digital	3 kondisi	compensation success, terminal-state coverage

2 EDA = tanpa dan dengan Proteksi. Tiga kondisi dibandingkan hanya pada F4b, F5, F6, dan tanpa gangguan; F1-F4a hanya EDA vs EDA; tanpa skor gabungan.

3.330 run: 37 kombinasi × 3 tingkat beban × 30 replikasi



SATU RUN



Hasil berlaku pada parameter uji, bukan klaim arsitektur terbaik

Batasan

- Data sintetis dan beban simulasi
- Satu host (Docker Compose)
- Gangguan F1-F6; tanpa partisi permanen dan split-brain
- Di luar lingkup: aspek medis, FEFO, stock opname

Luaran

- Analisis komparatif tiga arsitektur
- Bukti trade-off: konsistensi, performa, kompleksitas
- Bahan pertimbangan migrasi Apotek Bisma, terpisah dari keputusan produksi

Terima kasih

Mohon masukan dan arahan Bapak/Ibu Penguji.

Cadangan: parameter eksperimen dan peran tiap angka

Parameter	Nilai	Peran dalam eksperimen
Beban sistem	10, 50, 100 req/s	Tiga tingkat tekanan; bukan 450 transaksi per hari
Konkurensi F5	20 pembaruan serentak	Intensitas kompetisi pada satu stok
Replikasi	30 per konfigurasi	Pengulangan tiap konfigurasi uji
Warm-up dan recovery window	60 dtk dan 60 dtk	Pemanasan sistem; batas pemulihan setelah gangguan
Batas waktu transaksi	30 dtk sejak reservasi	Batas bisnis pembayaran (F4b, QRIS Statis)
Retry dan DLQ	5 kali: 1, 2, 4, 8, 16 dtk	Kebijakan pesan gagal, lalu masuk DLQ
Konkurensi consumer	2 pemroses per layanan EDA	Disamakan pada kedua kondisi EDA
Penundaan dan penghentian	10 dtk (F4a; F1 dan F3)	Durasi gangguan yang sama pada tiap pengujian

Cadangan: definisi operasional yang dipertegas

Isu	Penegasan	Dasar di proposal
Titik terlambat (F4b)	Terlambat bila received_at, saat sistem pertama menerima notifikasi (sebelum broker), melebihi reservasi + 30 dtk. Sama di 3 kondisi. Status final tidak berubah walau diproses belakangan.	Tabel 3.4 dan 3.2.3; titik acuan baru [R2]
Tanpa Proteksi vs dasar	Dasar sama di kedua EDA: publisher confirm, durable, ACK manual, 2 consumer. Paket: Outbox, Inbox, OCC, Saga, Retry dan DLQ.	3.2.2 dan Tabel 3.4 [R3]
Kriteria pilot	Run selesai sampai rekonsiliasi; request dan event terlacak lewat ID; skrip tanpa error; hasil transaksi = validasi fungsional awal, bukan data utama.	3.3.1 dan Tabel 3.7 [R4]
Bukti Retry dan DLQ	Dicatat dari log: percobaan ulang, pesan masuk DLQ, hasil pemulihan. Data pendukung tanpa ambang; kontribusi tunggal tidak disimpulkan.	Tabel 3.4 [R5]
Request vs transaksi	Request rate = laju kirim generator. Gagal teknis tetap beban; penolakan bisnis dipisah. Laporan request, transaksi sukses, penyebut.	Tabel 3.6 dan 3.3.3 [R6]
Pelaporan lintas arsitektur	3 kondisi hanya pada F4b, F5, F6, tanpa gangguan; F1-F4a hanya EDA vs EDA; per skenario dan beban, tanpa skor gabungan.	3.3.3 [R7]